

Міністерство освіти і науки України
Національний Університет “Львівська Політехніка”



Лабораторна робота
з дисципліни
“Об’єктно-орієнтоване програмування, частина 2”

Виконав:
студент гр. ІХ-22
Поліщук Юра

Прийняв:
Гордійчук-Бублівська О.В

Львів-2026

Мета:

Ознайомлення з основами моделювання телекомунікаційних мереж. Вивчення використання об'єктно-орієнтованого програмування для моделювання мережі. Моделювання різних топологій мереж (зірка, кільцева, деревоподібна, сіткова (mesh), комбінована тощо). Симуляція передачі пакетів між вузлами та маршрутизаторами з використанням TCP та UDP. Вимірювання продуктивності мережі: швидкість передачі, статистика втрат, затримки передачі. Побудова візуалізації для графічного відображення структури та роботи мережі.

Послідовність виконання:

- Вибір топології
- Кільцева (Ring)
- Створення класів вузлів мережі
- Визначте структуру класів Node, Router для відображення мережевих вузлів.
- Додайте методи для встановлення з'єднань між вузлами.
- Реалізація передавання пакетів
- Створіть клас Packet, що міститиме дані про відправника, отримувача, розмір пакета.
- Додайте симуляцію випадкової втрати пакетів (10-15% шанс втрати).
- Розробка моделей для TCP та UDP
- Реалізуйте класи TCPProtocol і UDPProtocol, які оброблятимуть пакети відповідно до правил кожного протоколу.
- Створення асинхронного механізму моделювання
- Використайте asyncio для моделювання мережевого трафіку.
- Заплануйте події передачі пакетів з випадковими затримками.
- Вимірювання продуктивності
- Обчисліть статистику для:
 - Часу передачі пакетів
 - Кількості втрачених пакетів
 - Пропускної здатності мережі
- Візуалізація мережі

- Використайте networkx для створення графічного представлення обраної топології.
- Позначте активні зв'язки між вузлами.
- Аналіз продуктивності
- Порівняйте результати для різних топологій.
- Зробіть висновки про ефективність мережевої топології на основі отриманих даних.

```

1  import asyncio
2  import random
3  import time
4  import networkx as nx
5  import matplotlib.pyplot as plt
6  import sys
7
8  class Node:
9      def __init__(self, name):
10         self.name = name
11         self.neighbors = []
12         def connect(self, other):
13             self.neighbors.append(other)
14
15  class RingSim:
16      def __init__(self, names):
17         self.names = names
18         self.nodes = {n: Node(n) for n in names}
19
20         for i in range(len(names)):
21             self.nodes[names[i]].connect(self.nodes[names[(i+1)%len(names)]])
22         self.stats = {"lost": 0, "times": []}
23
24         async def send(self, src, dst):
25             print(f"Sending packet from {src} to {dst}...", flush=True)
26             start = time.time()
27             await asyncio.sleep(random.uniform(0.3, 0.6))
28
29             if random.random() < 0.15:
30                 print(f"(!) Packet {src} -> {dst} LOST", flush=True)
31                 self.stats["lost"] += 1
32                 return
33
34             delay = time.time() - start
35             self.stats["times"].append(delay)
36             print(f"[*] Packet {src} -> {dst} delivered in {delay:.4f}s", flush=True)
37
38         def draw(self):
39             print("\nВиведення графіку... Закрийте вікно, щоб завершити.", flush=True)
40             G = nx.cycle_graph(self.names, create_using=nx.DiGraph)
41             plt.figure(figsize=(6,6))

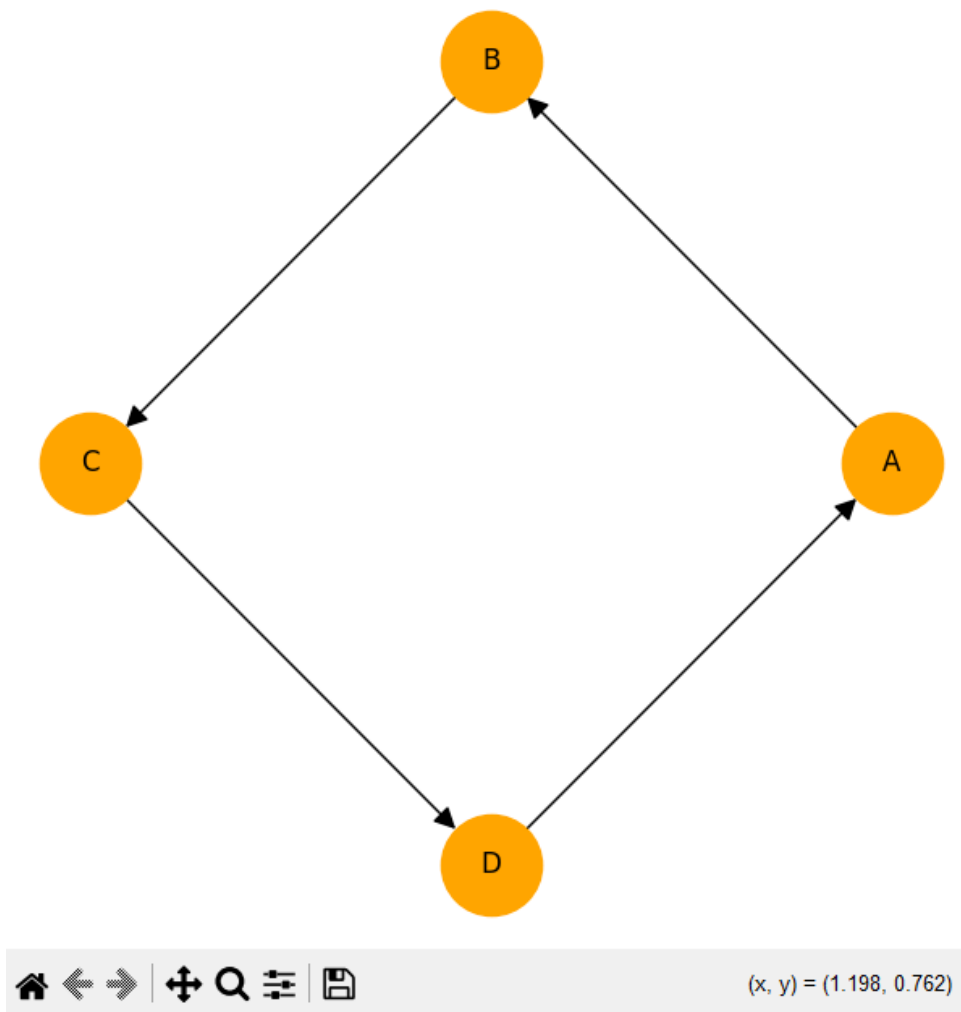
```

```

31
32     delay = time.time() - start
33     self.stats["times"].append(delay)
34     print(f"[*] Packet {src} -> {dst} delivered in {delay:.4f}s", flush=True)
35
36     def draw(self):
37         print("\nВиведення графіку... Закрийте вікно, щоб завершити.", flush=True)
38         G = nx.cycle_graph(self.names, create_using=nx.DiGraph)
39         plt.figure(figsize=(6,6))
40         pos = nx.circular_layout(G)
41         nx.draw(G, pos, with_labels=True, node_color='orange', node_size=2000, ar
42         plt.title("Network Topology: RING")
43         plt.show()
44
45     async def main():
46         names = ["A", "B", "C", "D"]
47         sim = RingSim(names)
48         tasks = [sim.send("A", "B"), sim.send("B", "C"), sim.send("C", "D"), sim.send
49         await asyncio.gather(*tasks)
50         avg = sum(sim.stats["times"])/len(sim.stats["times"]) if sim.stats["times"] e
51         print("\n" + "="*40, flush=True)
52         print("Performance Metrics for ring topology:", flush=True)
53         print(f"{{'total_packets': 4, 'lost_packets': {sim.stats['lost']}, 'average_t
54         print("="*40 + "\n", flush=True)
55         sim.draw()
56     if __name__ == "__main__":
57         asyncio.run(main())

```

Figure 1



На даній лабораторній роботі я Ознайомився з основами моделювання телекомунікаційних мереж. Вивчення використання об'єктно-орієнтованого програмування для моделювання мережі